

C++ ASSIGNMENT PROBLEM STATEMENTS

Inheritance, Runtime Polymorphism and RTTI

The following assignments are designed to provide hands-on practice with inheritance, function overriding, virtual functions, runtime polymorphism, abstract classes, base-class pointers and RTTI.

Problem 1: Hospital Billing System Using Inheritance, Polymorphism and RTTI

Domain: Hospital / Healthcare

Problem Statement

A hospital provides treatment to different categories of patients. The billing process differs depending on the type of patient and the services provided.

Develop a Hospital Billing System using object-oriented programming concepts in C++. The system should maintain patient information and calculate the final hospital bill according to the patient's category.

Create a base class named Patient that stores common details such as Patient ID, Patient Name, Age and Number of Days Admitted.

Derive InPatient and OutPatient from Patient. Further derive EmergencyPatient from InPatient.

An InPatient may be charged for room charges, doctor consultation, medicines and treatment. An OutPatient may be charged for consultation, diagnostic tests and medicines. An EmergencyPatient should include inpatient charges along with emergency service charges and ambulance charges, if applicable.

The program should calculate and display the bill differently for each type of patient.

Requirements

- Create a base class Patient with common data members such as patientId, patientName, age and numberOfDaysAdmitted.
- Provide appropriate constructors to initialize patient information.
- Create virtual functions such as calculateBill() and displayPatientDetails().
- Derive InPatient and OutPatient from Patient, and derive EmergencyPatient from InPatient.
- Override calculateBill() in InPatient, OutPatient and EmergencyPatient.
- Use suitable billing formulas for each patient category.
- Maintain different patient objects using Patient pointers, such as an array or vector of Patient pointers.
- Demonstrate runtime polymorphism by calling calculateBill() and displayPatientDetails() through Patient pointers.
- Use RTTI with dynamic_cast or typeid to identify the actual runtime type of each patient object.
- If an object is an EmergencyPatient, display additional information such as emergency charges, ambulance charges and type of emergency.

- Perform safe casting by checking the result of `dynamic_cast` before accessing derived-class-specific members.
- Include a virtual destructor in the Patient class.
- Create a menu-driven application with options to add different patient types, display all patients, calculate bills and identify patient types using RTTI.
- Ensure that all dynamically created objects are properly deleted before program termination.

Problem 2: Financial Account Management System Using Inheritance, Polymorphism and RTTI

Domain: Finance / Banking

Problem Statement

A bank provides different types of accounts to its customers. Each account contains common information, but the rules for calculating returns, handling withdrawals, overdraft facilities and maturity amounts differ depending on the account type.

Develop a Financial Account Management System using inheritance, runtime polymorphism and RTTI.

Create a base class named Account containing common information such as Account Number, Customer Name and Balance.

Derive SavingsAccount, CurrentAccount and FixedDepositAccount from Account. A savings account earns interest based on its balance. A current account may provide an overdraft facility and apply service charges. A fixed deposit account earns returns based on principal amount, interest rate and deposit tenure.

The program should process all account types through a common base-class interface while also identifying the actual account type at runtime.

Requirements

- Create a base class Account with data members such as `accountNumber`, `customerName` and `balance`.
- Provide suitable constructors for initializing account information.
- Declare virtual functions `calculateReturns()` and `displayAccountInfo()`.
- Derive SavingsAccount, CurrentAccount and FixedDepositAccount from Account.
- Include additional data in SavingsAccount such as `interestRate` and `minimumBalance`.
- Override `calculateReturns()` to calculate interest earned by a savings account.
- Include `overdraftLimit` and `serviceCharges` in CurrentAccount and implement suitable withdrawal rules.
- Include principal amount, interest rate and deposit tenure in FixedDepositAccount and calculate the maturity amount.
- Store objects of all account types using Account pointers.
- Demonstrate runtime polymorphism by calling `calculateReturns()` and `displayAccountInfo()` through Account pointers.
- Use `dynamic_cast` to identify the actual runtime account type.

- For SavingsAccount, display interest rate and minimum balance.
- For CurrentAccount, display overdraft limit and service charges.
- For FixedDepositAccount, display interest rate, deposit tenure and calculated maturity amount.
- Perform safe downcasting by checking the result of dynamic_cast before accessing derived-class-specific members.
- Create a menu-driven application to create accounts, display accounts, calculate returns, perform withdrawals and identify account types using RTTI.